Dictionary Server Report

Han Perry 693878

Introduction

This report describes the design and implementation of `DictionaryServer` and `DictionaryClient`. The server is a multi-threaded application that provides a dictionary service to clients, allowing a client to query the meaning(s) of a word, add a new word, remove an existing word, add a new meaning to a word and additional features such as editing and removing a selected meaning from a word. The server is implemented in Java and uses sockets for communication with clients. The server is capable of handling multiple clients concurrently through multithreading. The server uses a MySQL database to store the dictionary data. On the client side, there is a user-friendly graphical user interface.

Sockets and Networking

Internet Protocol

The server and client communicate using the Transmission Control Protocol (TCP), chosen for its reliability and guarantee of lossless message arrival [3].

Server

The server initialises a listening socket set to the port number specified by the user, and listens for incoming client connections. When a client connect, a `WorkerRunnable` object is created with the accepted client socket and added to the threadpool queue to be executed. Once processed, the worker thread sends a response back to the client and closes the connection.

Client

The `ClientConnect` class handles the client-side socket connection. For each action (e.g. a word query), the client will establish a new connection with the server. Because the actions performed are fairly simple, this short-lived connection is an acceptable design choice and decreases the risk of an accidental disconnection. The overhead of establishing a new connection for each action should be reasonable as the client is not expected to perform a large number of actions in quick succession. When the client sends a request to the server it then waits for a response, which acts as a way to reduce the risk of spam actions from a single unmodified client.

Since both client and server use the `writeUTF` and `readUTF` methods, the dictionary is compatible with unicode characters, including emoji.

Concurrency and Multithreading

Architecture

The server utilises the Java Executor Framework [6] to manage a thread pool of worker threads, allowing multiple clients to connect concurrently. The server uses `ThreadPoolExecutor` to create a customised thread pool architecture, based on a cached thread pool [5] but with additional constraints of a minimum and maximum numbers of threads. When required, new threads are created up to the maximum number of threads, previously constructed threads are reused when available, and idle threads are terminated after a specified time (set to 60 seconds). A small core pool size of five is also set as a way to maintain some threads in the pool even when idle. This architecture allows the server to handle a usual load of clients concurrently with reduced overhead for thread creation, while also being able to scale up to meet increased demand. A maximum thread count of 20 is set to prevent resource exhaustion in the case of a large number of clients connecting simultaneously. Although this maximum protects against resource exhaustion, it limits the server's ability to handle a large number of clients concurrently if the actions requested are long-running. However, this is an acceptable trade-off as the server is designed for simple dictionary tasks that generally do not take a long time to process.

Message Protocols

The server and client communicate using the `Message` class, which is a data structure that contains the information for the selected action. The `command` is a string that specifies the action that the client is requesting or the action request that the server is responding to. The `word` is the string word that the action is being carried out on, and `meaning` and `oldMeaning` are both single meanings referring to the word (the former being used in a message where it is requesting for an existing meaning to be edited). The `meanings` arraylist is used to send the multiple meaning of a word. For error handling there is an `error` string that is used to send details of the issue back to the client if an error occurs, `success` for a boolean indicator of success or failure, and `valid` for message format validation. The `Message` class is shown in Figure 1.

The message is serialised using the `writeUTF` and `readUTF` methods of the `DataOutputStream` and `DataInputStream` classes, respectively. The message protocol is simple and easy to understand, making it suitable for the simple dictionary tasks that the server is designed to handle.

Dictionary Implementation

Used mySQL database for dictionary storage and Java JDBC for interacting with this database [2].

Graphical User Interface

theme: [4]

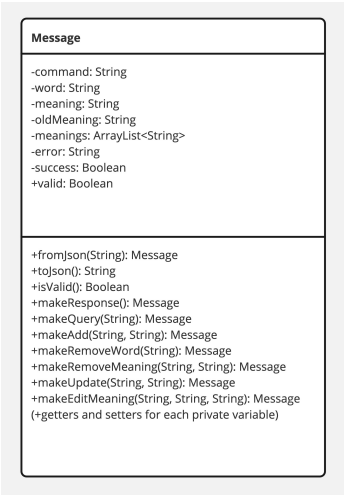

Figure 1: Message class

Error Handling

Logging

Used log4j2 for logging [1].

Excellence

Used Maven for dependency management with a multi-module project structure [7].

Creativity

References

[1] Apache. *Apache Log4j*. 2024. URL: `https://logging.apache.org/log4j/2.x/` (visited on 03/06/2024).

[2] Vogella Blog. *MySQL and Java JDBC - Tutorial*. 2024. URL: `https://www.vogella.com/tutorials/MySQLJava/article.html` (visited on 03/30/2024).

[3] W. Eddy. *RFC 9293: Transmission Control Protocol (TCP)*. USA, 2022.

[4] Formdev. *Flatlaf Themes*. 2024. URL: `https://www.formdev.com/flatlaf/themes/` (visited on 04/07/2024).

[5] GeeksforGeeks. *Thread Pools in Java*. 2024. URL: `https://www.geeksforgeeks.org/thread-pools-java/` (visited on 03/30/2024).

[6] Oracle. *Concurrent Package Summary*. 2024. URL: `https://docs.oracle.com/javase/8/docs/api/java/util/concurrent/package-summary.html` (visited on 03/30/2024).

[7] Apache Maven Project. *POM reference*. 2024. URL: https://maven.apache.org/pom.html (visited on 03/30/2024).